

* Spring MVC :- Spring MVC is the official web framework for building web applications within the Spring Framework. It allows developers to create both REST APIs (returning JSON) and traditional Web Applications (returning HTML or JSP views).

→ Relation with Spring Boot :-

- Spring MVC provides the core annotations and functionality (like @Controller, @RequestMapping, @PostMapping, etc.).
- Spring Boot acts as an auto-configuration layer over Spring MVC, abstracting away complex boilerplate code like server setup and dependency management.

→ Core Flow :-

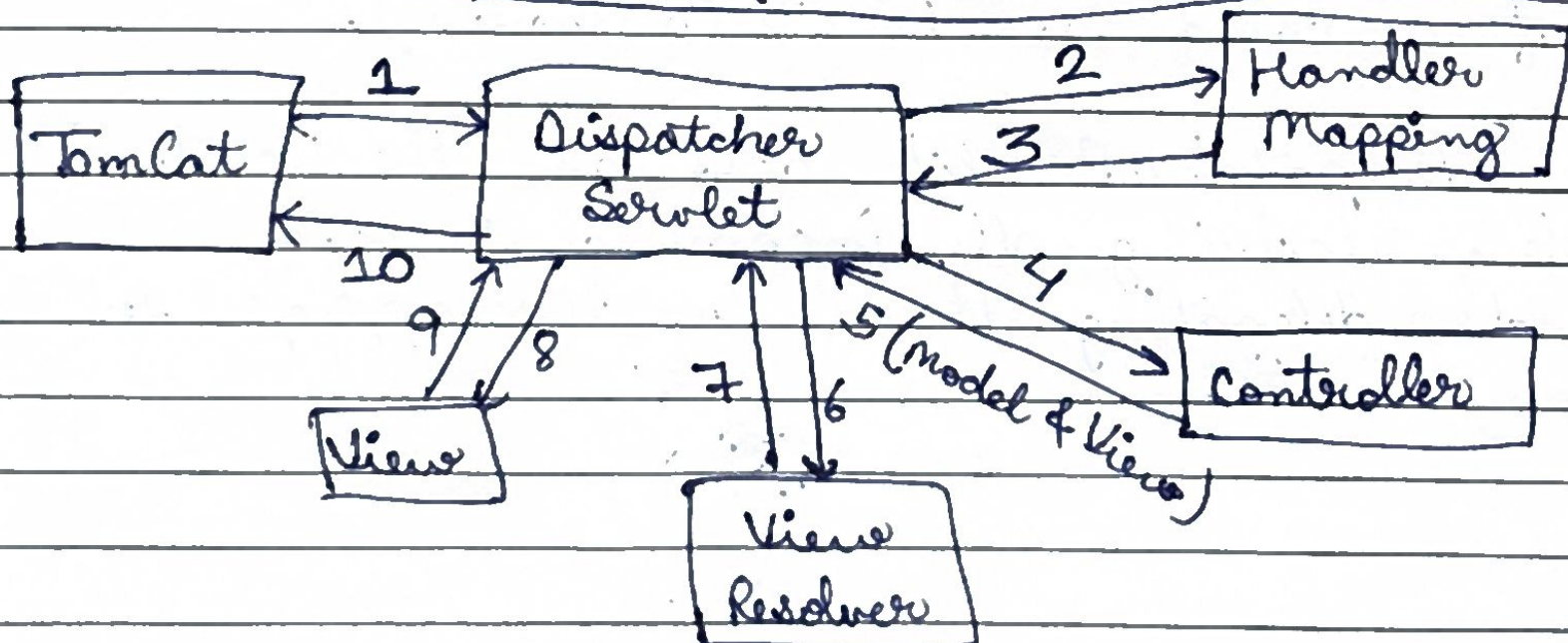
- When a client sends a request (e.g. GET /hello), it first hits the 'DispatcherServlet' (often called as 'Front Controller').
- The DispatcherServlet acts as the central hub, receiving all incoming requests and delegating them to the appropriate components.

→ Key Components :-

1. HandlerMapping → This component scans your code (after the IOC Containers Initialises) for annotations like `@PostMapping` or `@GetMapping` to create a "mapping table". It determines which controller method should handle a specific endpoint.

2. HandlerAdapter → Once the `DispatcherServlet` knows which method to call via the `HandlerMapping`, the `HandlerAdapter` is used to physically invoke that method.

* Dispatcher Servlet :- It is the central component of Spring MVC that receives all HTTP requests, forwards them to appropriate controllers, and manages the response generation process including handler mapping and view resolution.



Date.....

Clients generally share information with the server in three ways:-

1. Request Body → Typically used in POST or PUT requests (often JSON), which the `DispatcherServlet` converts into a Java object using a library like 'Jackson'.

2. Path Variables → Data embedded directly in the URL path (e.g., `/users/1`), where 1 represents an ID.

3. Query/Request Parameters → Data sent as part of the URL string after a question mark. (`users?id=1`).

Unlike traditional Servlet-based applications where you manually handle these parameters in every single servlet, the "`DispatcherServlet`" automates this mapping, allowing you to use simple annotations like `@RequestBody`, `@PathVariable`, and `@RequestParam` within your controller methods to access this data seamlessly.

* View Layer :-

Beyond REST APIs, Spring MVC supports traditional web applications that render pages directly to the browser. This is where the Model-View-Controller (MVC) pattern comes into play.

- Model → An object used to carry data from the controller to the view.
- View → The UI layer, which can be an HTML, or a JSP (Java Server Pages) file. JSP allows you to embed Java code inside HTML for dynamic content.
- Controller → Handles the HTTP request and returns the name of the "view" to be rendered.

→ View Resolution Process :-

1. View Resolver → Because the controller returns just a string (e.g. "home"), the View Resolver is responsible for turning that string into a physical file path (e.g. "home.jsp") using prefixes and suffixes.

Date.....

2. Boilerplate → In a standard Spring MVC project, you manually configure the ViewResolver and dependencies to enable this flow.

*** Unlike Spring Boot, where configuration is abstracted, in Spring MVC, you must handle these things first:-

- Dependencies should be manually included in `pom.xml` for "Spring Web MVC", "Embedded Tomcat" and "Jackson".
- Main Config → You need to start embedded server and manually configure 'DispatcherServlet'. This includes setting up 'ContextPath' and a physical base directory (webapps) for application.
- IOC Container Initialization → You must use the `AnnotationConfigWebApplicationContext` to support web-specific features.

We have to write this much boilerplate.

But, Spring Boot effectively automates these tedious configurations, allowing you to focus entirely on business logic (like your controllers & service layers) rather than infrastructure setup.